Splitwise Debt Simplifier

Cash Flow Minimizer & Settle Simulator Utility

Project Metadata

Author: Tuba Mirza

Email: tubamirza822@gmail.com

Languages: C++ (Core Library) & JavaScript (Web UI)

Live Website: https://mirzasayzz.github.io/debt-simplification/

Summary: This document outlines the requirements, architectural design, hybrid algorithm implementation, and validation procedures for the Splitwise Debt Simplifier project.

1. Introduction & The Requirement (The Ask)

In typical group outings or trips, friends make expenses on behalf of one another. Recording these transactions creates a complex network of debt. For example, if Alice owes Bob, and Bob owes Charlie, these intermediate payments are redundant. Alice can pay Charlie directly.

The objective of this project is to simplify all group debts and determine the minimum number of transactions required to settle all balances completely.

Project Requirements Checklist:

- Every person's final balance must eventually become zero.
- The total net amount paid and received must remain mathematically correct.
- The total number of money transfers (transactions) is minimized.
- Safe from integer overflows: Handles large amounts up to 10^9 per transaction.
- Dual Interfaces: Includes both a high-performance C++ backend and a visual, easy-to-use Web UI.

2. Mathematical Strategy & Hybrid Algorithm

Finding the absolute minimum number of transactions is an NP-hard problem, reducible from the Subset Sum / Partition problem. A group of K people whose balances sum to exactly 0 can be settled in K - 1 transactions. To minimize transactions, we must maximize the number of independent zero-sum subgroups.

To solve this efficiently, we implemented a hybrid algorithm that automatically selects the best solver based on the size of the active participant group (N):

A. Optimal Bitmask Dynamic Programming (For N <= 20)

When the number of active participants is 20 or less, the system runs a Dynamic Programming algorithm using bitmasks. It computes the maximum number of zero-sum subsets in O(2^N * N) time. Once subgroups are isolated, they are settled sequentially in K - 1 transactions, guaranteeing the mathematically absolute minimum number of transactions.

B. Greedy Priority Queue Solver (For N > 20)

For larger groups, the exponential DP time complexity is too slow. The solver automatically falls back to a greedy heuristic. It loads all creditors into a Max-Heap and all debtors into a Min-Heap. At each step, it settles the largest debtor with the largest creditor. This runs in O(N log N) time and is highly scalable.

3. Implementation Overview

The project is structured with dual implementations (C++ for native performance, and JavaScript for the Web UI) sharing the same algorithmic core.

Directory Layout:

```
pea/
|-- index.html                    # Web UI main entry point
|-- style.css                     # Premium dark-mode glassmorphic styling
|-- app.js                        # JavaScript ported algorithm & DOM controller
|-- include/
|   `-- minimize_cash_flow.h      # C++ library function declarations
|-- src/
|   |-- minimize_cash_flow.cpp    # Core C++ Hybrid Algorithm implementation
|   `-- main.cpp                  # CLI runner with preset demos
|-- tests/
|   `-- test_minimize_cash_flow.cpp # Automated C++ test suite
|-- Makefile                      # C++ build automation script
`-- .gitignore                    # Git file exclusions
```

Web UI Features:

1. Settle Up Button: Instantly solves the entered expenses list.
2. Step-by-Step Settlement Simulator: A single action button ('Next Step') that animates a coin transfer and dynamically counts down the debtor's and creditor's balances.
3. How It Was Calculated Log: A monospaced execution log detailing name-to-index registration, subset sum computations, and settle transitions.
4. Independent Groups Display: Lists the isolated zero-sum groups found.

4. Verification Results & Deployment

C++ Unit Testing:

An automated C++ test suite is included in tests/test_minimize_cash_flow.cpp. It compiles and runs via 'make test'. Six critical scenarios are verified, including chain debts, circular debts, self-debts, zero values, long-value overflows, and large-scale greedy runs. All tests compile and pass successfully.

Deployment:

The Web UI is deployed using GitHub Actions pages-build-deployment workflow. The code is fully committed and live at https://mirzasayzz.github.io/debt-simplification/.

Status: Verified & Live